

Recommandations générales
Édition/Exécution. Un code python se manipule au choix : · dans un éditeur de texte brut quelconque ; l'exécution est obtenue en ligne de commande : > python3 moncode.py · dans un IDE (Integrated Development Environment) comme Pyzo ou Spyder ; il intègre le noyau d'exécution python et des outils de développement (documentation / état des variables / débog) · dans un notebook Jupyter qui permet d'organiser les éléments de code dans des cellules d'un document enrichi et de les y exécuter.
Debuguage. Une erreur ne peut se résoudre qu'après l'avoir identifiée et comprise ! Quelques remarques : · si le noyau python renvoie une erreur, alors lisez-la ! · un <code>print(x)</code> glissée dans un programme permet d'afficher la valeur de la variable <code>x</code> · les variables sont « typées » et certaines syntaxes sont spécifiques à un type donné. Utiliser <code>type(x)</code> pour identifier le type de <code>x</code> · l'indentation et l'utilisation des : dans les structures de contrôles est très importante. Lorsque la syntaxe est correcte mais que le résultat obtenu est faux, une stratégie de debuguage efficace consiste à décliner une série de tests <u>élémentaires</u> et <u>progressifs</u> qui permettent de <u>valider</u> les différentes étapes de l'algorithme indépendamment les unes des autres.

Environnements à utiliser	
<code>from math import *</code>	importe toutes les fonctions mathématiques du module <code>math</code>
<code>import numpy as np</code>	importe le module qui manipule les vecteurs et matrices
<code>import matplotlib.pyplot as plt</code>	importe le module qui trace les courbes

Quelques fonctions et variables
<code>sqrt</code> , <code>log</code> , <code>exp</code> , <code>cos</code> , <code>sin</code> , <code>tan</code> , <code>cosh</code> , <code>sinh</code> , <code>tanh</code> , <code>abs</code> , <code>floor</code> , <code>round</code> Il faut rajouter <code>np.</code> devant les fonctions afin de les appliquer à des vecteurs composante par composante. <code>x.real</code> et <code>x.imag</code> renvoient la partie réelle et la partie imaginaire de l'élément <code>x</code> <code>pi</code> , <code>e</code> , <code>1j</code> (pour le complexe <i>i</i>)

Booléens et opérateurs logiques	
<code>True</code> , <code>False</code> <code>==</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code> , <code>!=</code> <code>and</code> , <code>or</code> , <code>not</code>	Booléens VRAI et FAUX, respectivement Tests <code>=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code> , <code>≠</code> respectivement Opérateur logique ET, OU, NON respectivement

Structures de contrôle	
<code>def f(x1, x2):</code> <code>...</code> <code>return y1, y2</code>	Fonction <code>f</code> d'arguments <code>x1</code> et <code>x2</code> et qui renvoient <code>y1</code> et <code>y2</code> .
<code>for i in range (a,b,p):</code> <code>...</code>	Boucle pour <code>i</code> allant de <code>a</code> à <code>b-1</code> avec le pas <code>p</code> . (<code>p = 1</code> par défaut)
<code>while condition :</code> <code>...</code>	Boucle tant que la condition est vraie (avec condition un booléen)
<code>if cond1 :</code> <code>...</code> <code>elif cond2 :</code> <code>...</code> <code>else :</code> <code>...</code>	Conditionnelle. On peut utiliser plusieurs <code>elif</code> successifs. <code>elif</code> est la contraction de <code>else if</code> du langage courant.

Définition de vecteurs et matrices	
<code>np.array([2,5,7])</code> <code>np.array([[1,2],[4,5]])</code> <code>np.arange(a,b,p)</code> <code>np.linspace(a,b,N)</code> <code>np.zeros(n)</code> <code>np.zeros((m,n))</code> <code>np.ones(n)</code> <code>np.ones((m,n))</code> <code>np.eye(m,n)</code> <code>np.diag(X)</code> <code>np.diag(X,i)</code>	Vecteur (ou tableau) ${}^t(2, 5, 7)$ Matrice $\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix}$ Vecteur des valeurs entières allant de <code>a</code> à <code>a+kp</code> avec le pas <code>p</code> (<code>p = 1</code> par défaut) et où <code>a+kp</code> est le plus proche de <code>b</code> par valeur strictement inférieure à <code>b</code> Vecteur de <code>N</code> valeurs équiréparties entre <code>a</code> et <code>b</code> (inclus) Vecteur nul de taille <code>n</code> Matrice nulle de taille <code>m×n</code> Vecteur de 1 de taille <code>n</code> Matrice de 1 de taille <code>m×n</code> Matrice de type identité de taille <code>m×n</code> Construction de la matrice carrée dont la (0-ième) diagonale est le vecteur <code>X</code> Construction de la matrice carrée dont la <code>i</code> -ième diagonale est le vecteur <code>X</code>

Manipulation de vecteurs et matrices	
<code>len(X)</code> <code>np.shape(A)</code> ou <code>A.shape</code> <code>np.size(A)</code> ou <code>A.size</code> <code>np.shape(A)[0]</code> <code>np.shape(A)[1]</code> <code>A[a:b,A:B]</code> <code>A[:,0]</code> <code>A[1:-1,0]</code> <code>np.diag(A)</code> <code>np.diag(A,i)</code> <code>A+x</code> <code>A#B</code> <code>A.dot(B)</code> <code>np.transpose(A)</code> ou <code>A.T</code>	Longueur d'un vecteur (ou nombre de lignes d'une matrice <code>X</code>) Taille <code>m×n</code> de la matrice <code>A</code> Nombre d'éléments de la matrice <code>A</code> Nombre de lignes de la matrice <code>A</code> Nombre de colonnes de la matrice <code>A</code> Extraction de la matrice composée des lignes <code>a</code> à <code>b-1</code> et des colonnes <code>A</code> à <code>B-1</code> Extraction de la première colonne de <code>A</code> Extraction de la première colonne de <code>A</code> excepté le premier et le dernier coefficient Extraction du vecteur formant la diagonale de <code>A</code> Extraction du vecteur formant la <code>i</code> -ième diagonale de <code>A</code> Rajoute la valeur <code>x</code> à tous les éléments de <code>A</code> où l'opérateur binaire <code>#</code> ∈ { <code>+</code> , <code>*</code> , <code>.</code> , <code>/</code> , <code>**</code> }, Opération composante par composante entre les <code>np.array</code> <code>A</code> et <code>B</code> Produit matriciel de <code>A</code> avec <code>B</code> (si <code>A</code> et <code>B</code> sont des <code>np.array</code>) Transposée de la matrice <code>A</code>

Opérations d'algèbre linéaire	
<code>np.linalg.norm(X,p)</code> <code>np.linalg.norm(A,p)</code> <code>np.linalg.det(A)</code> <code>np.linalg.matrix_rank(A)</code> <code>np.trace(A)</code> <code>np.linalg.inv(A)</code> <code>np.linalg.solve(A,b)</code> <code>np.linalg.eigvals(A)</code> <code>np.linalg.eig(A)</code> <code>np.vdot(X,Y)</code>	$\ X\ _p$ ($p \in [1, +\infty[$ ou $p = \text{inf}$) $\ A\ _p$ (norme subordonnée lorsque $p \in \{1,2,\text{'inf'}\}$, de Frobenius si $p = \text{'fro'}$) Déterminant de la matrice <code>A</code> Rang de la matrice <code>A</code> Trace de la matrice <code>A</code> Inverse de la matrice <code>A</code> Résolution d'un système linéaire $Ax = b$ Valeurs propres de la matrice <code>A</code> (avec multiplicité) Couple contenant les valeurs propres et les vecteurs propres de la matrice <code>A</code> (vecteurs propres rangés en colonne) Produit scalaire entre les vecteurs <code>X</code> et <code>Y</code>

L'exemple suivant utilise très peu de fonctionnalités intégrées au logiciel et est plutôt un prétexte pour illustrer les usages de syntaxe et les possibilités de vectorisation.

Un problème d'interpolation

Étant données différentes valeurs d'un entier $n \in \mathbb{N}$, on souhaite éprouver la qualité de l'interpolation de Lagrange de la fonction $f : x \mapsto (1+25x^2)^{-1}$, en $n+1$ points uniformément répartis dans l'intervalle $[-1, 1]$: $x_j = j/n$, pour $0 \leq j \leq n$.

Le polynôme d'interpolation est calculé sous sa forme de Newton :

$$p(X) = f(x_0) + f[x_0, x_1](X - x_0) + \dots + f[x_0, x_1, \dots, x_n](X - x_0) \dots (X - x_{n-1}),$$

pour laquelle les coefficients sont obtenus en calculant récursivement les différences divisées (généralisation du taux d'accroissement aux ordres supérieurs) :

$$f[x_0, x_1, \dots, x_j] = \frac{f[x_1, x_2, \dots, x_j] - f[x_0, x_1, \dots, x_{j-1}]}{x_j - x_0}.$$

Le polynôme p est alors évalué par exponentiation rapide (algorithme de Horner).

```
1 from math import *
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5
6 def f(x):
7     y = 1/(1+25*x**2)
8     return y
9
10
11 def diffddiv(x,y):
12     n = len(x)
13     D = np.zeros((n,n))
14     D[:,0] = y
15     for j in range (1,n):
16         for i in range (n-j):
17             D[i,j] = (D[i+1,j-1]-D[i,j-1])/(x[i+j]-x[i])
18     return D
19
20
21 def fastexp(x,D,t):
22     n = len(x)
23     y = D[0,n-1]*np.ones(len(t))
24     for k in range (n-2,-1,-1):
25         y = (t-x[k])*y + D[0,k]
26     return y
27
28
29 x = np.linspace(-1,1,400)
30 y = f(x)
31 plt.plot(x,y, label = "f")
32
33 for n in range (1,10):
34     X = np.linspace(-1,1,n+1)
35     val = f(X)
36     D = diffddiv(X,val)
37     yp = fastexp(X,D,x)
38     plt.plot(x,yp,'--',label = "n = "+str(n))
39 plt.legend(loc="best")
40 plt.title("Polynomes d'interpolation de degre n")
41 plt.show()
```

